



GritAI Studio • Best Practices

The Boris *Way*

Anthropic's official best practices combined with Boris Cherny's personal workflow for getting the most out of Claude Code.

Last updated May 11, 2026

Context

Verification

Plan Mode

CLAUDE.md

Skills

Subagents

Cheat Sheet

The full guide in 13 takeaways. Scroll down for details.

01 • CONTEXT

Context is *king*

Everything comes back to the 200k-token window. Keep it clear, compact, and relevant.

02 • VERIFICATION

Give Claude a way to *verify*

Tests, linters, type-checkers. Self-correcting loops are the highest-leverage thing you can add.

03 • WORKFLOW

Explore. Plan. *Implement*. Commit.

Four steps, every time. Read first, plan second, implement third, then lock it in with a commit.

04 • PROMPTING

Be *specific*

Name files, describe behavior, point to patterns. Vague prompts waste tokens on guessing.

05 • INPUT

Feed Claude *rich* content

Use @-references, paste images, pipe in data. The more context you provide upfront, the better the output.

06 • MEMORY

CLAUDE.md is your project *brain*

Run /init, then maintain it. Style rules, architecture, common commands. Loaded every session.

07 • SKILLS

On-demand *expertise*

Markdown files with instructions that load only when invoked. Keeps CLAUDE.md lean.

08 • SUBAGENTS

Isolated context

Spawn child agents for focused tasks. They get their own context window and report back.

09 • SPEC GENERATION

Let Claude *interview* you

Ask Claude to ask you questions before coding. The spec it produces is better than what you'd write alone.

10 • DEBUGGING

Common *failure* patterns

Kitchen-sink sessions, repeating corrections, sunk cost. Know when to start fresh.

11 • SESSION CONTROL

Manage your *session*

Esc to stop, /clear to reset, /compact to shrink context, --continue to resume.

12 • ADVANCED

Pro *tips*

CLI tools, hooks, /permissions, emphasis in CLAUDE.md. Small techniques that compound over time.

13 • THE META-LESSON

Develop your *intuition*

The real skill is knowing what to delegate and when to step in. That takes practice, not rules.

THE ONE CONSTRAINT

Context is *king*

Everything Claude Code does well or poorly comes back to one thing. How much context does it have, and how relevant is that context? Every technique in this guide is ultimately a strategy for managing the 200k-token context window.



Be clear

Ambiguous prompts waste tokens on exploration. A specific prompt with file paths and expected outcomes gets Claude to the right place immediately.



Keep it compact

Use `/compact` between subtasks to summarize and free up space. Long conversations decay in quality as older context gets compressed.



Parallelize

Boris runs 5–10 parallel sessions at once. Plan in one, review in another. Use 3–5 Git worktrees or checkouts for separate branches. Think of AI as capacity you can schedule, not a single conversational tool.

```
# Track context usage in your statusline (always visible)
```

```
> /statusline
```

```
Customize your status bar to always show context usage
```

```
# Clear between unrelated tasks
```

```
> /clear
```

```
✓ Context cleared. CLAUDE.md reloaded.
```

*"Think of AI as **capacity you can schedule**, not a single conversational tool."*

Boris Cherny, creator of Claude Code (paraphrased)

HIGHEST LEVERAGE

Give Claude a way to *verify*

The single highest-leverage thing you can do is tell Claude how to check its own work. Without verification, Claude writes code and hopes. With it, Claude writes code, tests it, and iterates until the tests pass.

WITHOUT VERIFICATION

One shot, hope it works

- ✗ Claude writes code, then stops
- ✗ Bugs only show up when you test manually
- ✗ You spend time fixing what Claude missed

WITH VERIFICATION

Self-correcting loop

- ✓ **Claude writes, tests, and fixes iteratively**
- ✓ **Errors caught before you ever see them**
- ✓ **Dramatically higher first-attempt success rate**

Better prompts with verification built in

VAGUE

"Add a login page"

WITH VERIFICATION

"Add a login page. Write tests for email validation, wrong password,

and successful login. Run the tests and make sure they pass."

VAGUE

"Refactor the database module"

WITH VERIFICATION

"Refactor the database module to use connection pooling. Run `npm test` after each change. All 47 existing tests must still pass."

VAGUE

"Fix the CSS layout"

WITH VERIFICATION

"Fix the sidebar overlapping the main content on mobile. Verify by checking that no element wider than 100vw exists using `document.querySelector` in the browser console."

Anthropic's official recommendation: Give Claude tests, type-checks, linters, or other verification commands. Tell Claude to run them and iterate. This is the most impactful thing you can do for code quality.

The "go fix it" pattern: When something breaks, don't over-explain. Just say "go fix it" and trust the verification loop you set up. Claude fixes most bugs by itself when it has tests to run against. Boris uses the Claude-in-

Chrome extension daily for UI testing, iterating until it looks right and functions correctly.

THE WORKFLOW

Explore. Plan. *Implement.* Commit.

Anthropic recommends this four-step workflow for non-trivial tasks. Boris takes it further with a strict rule: always start complex tasks in plan mode, no exceptions. Each step constrains the next one, preventing Claude from going off-track.

1 Explore

Ask Claude to read the relevant code and understand the current state. Don't ask it to change anything yet. This grounds the conversation in facts rather than assumptions.

```
● ● ● ~/my-project
```

```
> Read src/auth/ and explain how the login flow works today.  
Don't change anything yet.
```

2

Plan

Ask Claude to propose a plan. Review it. This is where you catch misunderstandings before they become wrong code. Press **Shift+Tab** twice to enter plan mode for read-only guarantees. Boris iterates here until the plan is solid, then switches to auto-accept for implementation.

 ~/my-project

```
> Now write a plan for adding OAuth. Which files need to change?  
What's the order of operations? Don't implement yet.
```

3

Implement

Now Claude executes the plan. Because it already understands the code and you've agreed on the approach, implementation is faster and more accurate. Include verification.

 ~/my-project

```
> Implement the plan. After each file, run the test suite.  
Don't move on until all tests pass.
```

Let Claude write the commit message. It has seen every change and can summarize them accurately. If the diff is large, ask Claude to break it into logical commits.

```
● ● ● ~/my-project
```

```
> Commit this with a clear message. If the changes are large,  
split into separate logical commits.
```

```
✓ Created 2 commits:
```

```
a3f2c1d feat: add OAuth provider configuration
```

```
b7e4a9f feat: integrate OAuth into login flow with tests
```

*"Claude typically **executes in one shot** after a good plan. The planning prevents mid-course corrections that waste time and context."*

Boris Cherny (paraphrased)

PROMPTING

Be *specific*

Vague prompts produce vague results. The more specific you are about scope, sources, patterns, and symptoms, the better Claude performs. Think of it like briefing a new contractor: they're smart, but they need context to do good work.

Scope

TOO BROAD

"Improve the API"

SCOPED

"Add rate limiting to POST /api/users. 100 requests per minute per IP. Return 429 with retry-after header."

Sources

UNANCHORED

"Write a migration script"

ANCHORED

"Write a migration from the old schema in `db/v1.sql` to the new one in `db/v2.sql`. Handle the `user.role` column becoming a separate roles table."

Patterns

NO GUIDANCE

"Add error handling"

WITH PATTERNS

"Add error handling following the pattern in `src/api/orders.ts`. Use the `AppError` class. Log to Sentry. Return standard error shapes."

Symptoms

MINIMAL

"The app crashes sometimes"

DETAILED

"The app crashes when loading the dashboard after login. Stack trace points to `useEffect` in `Dashboard.tsx:47`. Only happens when the user has no recent orders."

Use voice dictation. You speak three times faster than you type, and your prompts get way more detailed as a result. On macOS, hit the function key twice to activate. More detail generally means better output.

Make Claude your reviewer. Say "grill me on these changes and don't make a PR until I pass your test" or "prove to me this works." Have Claude

diff between main and your feature branch. You can also say "write a detailed spec first" before handing work off.

INPUT

Feed Claude *rich content*

Claude Code isn't limited to text prompts. You can reference files, paste images, and pipe data directly from other tools. The richer your input, the less Claude needs to guess.



@-references

Type `@filename` to add a file directly to context. Works with URLs too:

`@https://...` fetches and includes the page content.



Images

Drag-and-drop screenshots, mockups, or error messages into the prompt. Claude reads images natively. Paste a design and say "build this."



Pipes

Pipe output from any shell command directly into Claude.

`git diff |`
`claude` gives Claude the exact changes to review. Works with logs, test output, anything.

Pipe git diff for a code review

\$ git diff main | claude "Review this diff for bugs"

Pipe test failures for debugging

\$ npm test 2>&1 | claude "Fix the failing tests"

Pass a URL as context

> Read @<https://api.example.com/openapi.json> and generate a TypeScript client

MEMORY

CLAUDE.md: your project *brain*

CLAUDE.md is a markdown file that Claude reads at the start of every session. It's your project's persistent memory. Write it like onboarding notes for a new team member who happens to be extremely capable.

```
~/my-project

# Create a CLAUDE.md automatically (run inside a Claude session)
> /init
✓ Created CLAUDE.md with project overview

# Example CLAUDE.md content:
# My Project
Tech stack: Next.js 14, TypeScript, Prisma, PostgreSQL
Test cmd: npm test (vitest)
Lint cmd: npm run lint (eslint + prettier)
Style: Functional components, no classes. Zod for validation.
```

✓ Include

- ✓ Bash commands Claude can't guess
- ✓ Code style rules that differ from defaults
- ✓ Testing instructions and preferred test runners

✗ Leave out

- ✗ Anything Claude can figure out by reading code
- ✗ Standard language conventions Claude already knows
- ✗ Detailed API docs (link instead)

- ✓ Repo etiquette: branch naming, PR conventions
- ✓ Architectural decisions and common gotchas

- ✗ Info that changes frequently
- ✗ Long tutorials or file-by-file descriptions



Project root

`./CLAUDE.md`

lives at the project root. Every Claude session in this project reads it automatically.



User global

`~/.claude/CLAUDE.md`

applies to every project. Good for personal preferences like "always use TypeScript" or "commit in English."



Subdirectories

Place `CLAUDE.md` in any subdirectory. Claude picks it up when working in that directory. Useful for monorepos with different conventions per package.

Keep it under 500 lines. If Claude keeps ignoring a rule, the file is probably too long and the rule is getting lost. Prune ruthlessly. If Claude already does something correctly without the instruction, delete it or convert it to a hook.

CLAUDE.md can import files with @-references. Keep the main file clean and concise. Put details in separate files and reference them with

*"After every correction, end your prompt with **update your CLAUDE.md so you don't make that mistake again**. This turns every mistake into a permanent improvement."*

Boris Cherny's golden rule (paraphrased)

SPECIALIZATION

Skills: on-demand *expertise*

CLAUDE.md is always loaded. Skills are loaded only when you invoke them with a slash command. This distinction matters because it keeps your base context clean while giving you deep specialization on demand.

CLAUDE.MD

Always loaded

- ✓ Read at session start
- ✓ Project-wide conventions
- ✗ Uses context even when not needed

SKILLS

Loaded on demand

- ✓ Only loaded when you call `/skill-name`
- ✓ Deep expertise for specific tasks
- ✓ Zero context cost when not active

```
# A skill is just a SKILL.md file in .claude/skills/
```

```
$ cat .claude/skills/review.md
```

```
---
```

```
name: code-review
```

```
description: Review staged changes for issues
```

```
---
```

```
Review the staged changes. Check for:
```

1. Security issues (injection, auth bypass)
2. Performance regressions
3. Missing tests for new logic
4. Style guide violations per CLAUDE.md

```
# Invoke it
```

```
> /review
```

```
✓ Loaded skill: code-review
```

Two types of skills. Auto-invoked domain knowledge that Claude applies when working in relevant areas. And workflow skills you invoke with a slash command for repeatable multi-step processes. Built-in skills like `/docx`, `/xlsx`, `/pptx`, and `/pdf` handle document creation out of the box.

Boris's approach: Create your own skills and commit them to Git. Any pattern you repeat becomes a portable skill you can reuse across every project. The team even builds learning-focused skills, like a spaced

repetition skill where you explain your understanding and Claude fills gaps.

DELEGATION

Subagents: *isolated* context

Subagents are separate Claude instances that your main session spawns. Each one gets its own context window. When it finishes, only the result comes back to the parent. This is how you handle tasks that would otherwise blow up your context.



Isolated context

Each subagent gets a fresh 200k token window. Reading 50 files won't pollute your main conversation.



Parallel execution

Multiple subagents can run at the same time. Audit the frontend while the backend tests run. Results merge when both finish.



Compact results

Only the summary returns to the parent. A subagent might read 100k tokens of code but return a 2k-token audit report.

```
# Claude automatically creates subagents for complex tasks
```

```
> Audit all API endpoints for security issues. Check auth,  
input validation, and rate limiting across all routes.
```

```
⚡ Spawning 3 subagents...
```

```
Agent 1: Checking auth middleware
```

```
Agent 2: Auditing input validation
```

```
Agent 3: Reviewing rate limiting
```

```
✓ Audit complete. Found 3 issues:
```

```
HIGH /api/admin/users missing auth check
```

```
MED /api/upload no file size limit
```

```
LOW /api/search missing rate limiter
```

Boris's specialized subagents: He uses role-specific subagents. Code Simplifier for cleanup. Verify App for end-to-end testing. Build Validator to ensure builds pass. Code Architect for design decisions. Think of them as team members with distinct responsibilities. Append "use subagents" to any request where you want Claude to throw more compute at the problem.

SPEC GENERATION

Let Claude *interview* you

With the AskUserQuestion tool in Claude Code, you get a powerful technique for complex features. Have Claude interview you first. Start with a minimal prompt, and Claude asks targeted questions about things you might not have considered: implementation details, edge cases, trade-offs. Then it generates a spec it can implement.

> I want to add a notification system. Interview me about the requirements, then write a spec.

? What types of notifications do you need?

(email, push, in-app, SMS)

> Email and in-app for now. Push later.

? Should users be able to configure their preferences?

(per-type opt-out, frequency digests, quiet hours)

> Yes, per-type opt-out. No digests yet.

? What triggers notifications?

(user actions, system events, scheduled)

> New comments, assignment changes, and a daily summary.

✓ Spec written to docs/notification-spec.md

12 sections, 3 data models, 7 API endpoints defined

Start fresh after the interview. Once the spec is complete, start a new Claude session to execute it. Clean context, focused entirely on implementation. The spec becomes your input for the next session.

*"Claude asks about things you might not have considered. Technical implementation, UI/UX, edge cases, trade-offs. **You can use this for much more than coding.**"*

The interview technique, via the Claude Code team

Common *failure* patterns

Anthropic calls these out explicitly. When Claude Code produces bad results, it's usually one of these patterns. Each has a specific fix.

Kitchen sink session

You start with one task, ask something unrelated, go back to the first. Context fills with irrelevant information.

Use `/clear`
between
unrelated tasks

Correcting over and over

Claude does something wrong, you correct it, still wrong. Context gets polluted with failed approaches.

After 2
failures, `/clear`
and rewrite the
prompt

Overspecified CLAUDE.md

CLAUDE.md is too long. Claude ignores half of it because important rules get lost in the noise.

Prune
ruthlessly, or
convert to hooks

Trust-then-verify gap

Claude produces plausible-looking code that doesn't handle edge cases. You ship without checking.

Always provide verification.
Can't verify?
Don't ship.

Infinite exploration

You ask Claude to investigate without scoping it. Claude reads hundreds of files, filling the context.

Scope investigations narrowly, or use subagents

Sunk cost fallacy

You keep pushing a session that's going nowhere instead of starting fresh. Boris admits 10–20% of sessions fail.

Quick abandonment.
Parallel sessions hedge against dead ends.

Recovery technique: After a mediocre fix, try: "Knowing everything you know now, scrap this and implement the elegant solution." Claude often finds better approaches once it understands the problem space. The team also re-enters plan mode when Claude starts derailing.

SESSION CONTROL

Manage your *session*

These six commands give you precise control over your Claude Code session. Learning them is like learning keyboard shortcuts in your IDE. You'll use them constantly.

Esc

Interrupt

Stop Claude mid-action. Use when it's heading in the wrong direction. You can give new instructions immediately.

Esc Esc

Rewind menu

Double-tap Esc or use **/rewind** to open the rewind menu. Restore a previous conversation state or summarize from a selected point.

/clear

Reset context

Wipe the conversation and start fresh. CLAUDE.md gets reloaded. Use when the conversation has drifted too far.

/compact

Compress

Summarize the conversation to free up context. Pass instructions for controlled compaction:

/compact Focus on the API changes.

"undo that"

Revert changes

Tell Claude "undo that" and it will revert its changes. Works for quick corrections when Claude went in the wrong direction.

--continue

Resume session

Pass **claude --continue** to pick up where you left off. Context and tool state are restored from the last session.

ADVANCED

Pro *tips*

Smaller techniques that add up over time. None of these are essential on day one, but experienced users rely on them heavily.

Install CLI tools. GitHub CLI, AWS CLI, gcloud. Claude can use them for external services without rate limits. Also tell Claude about your linter and type checker in CLAUDE.md. "After editing TypeScript files, always run `tsc --noEmit`" turns the compiler into a verification loop.

Use `/init` on existing projects. Run `/init` even on projects you've had for years. It analyzes your codebase to detect build systems, test frameworks, and code patterns. Surprisingly good starting point for a CLAUDE.md.

Emphasize with **IMPORTANT and **bold**.** Claude responds to emphasis in CLAUDE.md. "IMPORTANT: never use var" or "you **MUST** use the AppError class" carries more weight than a casual mention. Use this sparingly for rules that really matter.

Post-tool-use hooks. Use hooks for automatic formatting. After write or edit, run your formatter to catch the last 10% without approval prompts. Any workflow you repeat becomes a hook candidate.

Use `/permissions`, not `--dangerously-skip-permissions`. Never skip permissions in production. Use `/permissions` to pre-approve specific safe commands instead. Also check out `/sandbox` for OS-level isolation. Sandboxing defines upfront boundaries rather than bypassing all checks.

Always use the latest model. Boris optimizes for cost per reliable change, not cost per token. The correction tax of weaker models' hallucinations costs more than the model itself. Opus gives significantly better first-attempt success rates.

Keep learning with Claude. Enable the explanatory or learning output styles in `/config` to have Claude explain the why behind its changes. Have Claude generate visual HTML presentations explaining unfamiliar code, or draw ASCII diagrams of new protocols.

THE META-LESSON

Develop your *intuition*

The techniques in this guide aren't a checklist. They're a starting point. The real skill is developing an intuition for when to explore vs. implement, when to compact vs. start fresh, when to use a subagent vs. handle it yourself. That intuition comes from practice, not from reading guides.

As Anthropic puts it: sometimes you should let context accumulate because you're deep in a complex problem. Sometimes skipping the plan is right because the task is exploratory. And sometimes a vague prompt is exactly what you need to see how Claude interprets the problem. Pay attention to what works for you. Start with one technique from this guide. Use it for a week until it becomes automatic. Then add another.

*"Over time, you'll develop intuition **no guide can capture.**"*

Anthropic's official best practices

*"Context window is your most important resource.
Verification is non-negotiable. These aren't opinions."*

This is from the team that built Claude Code."

RESOURCES

Keep *learning*

OFFICIAL

Anthropic Best Practices

The official best practices guide from Anthropic's documentation.

[Read more →](#)

DOCS

Full Claude Code Docs

Complete reference for all features, commands, and configuration options.

[Read more →](#)

VIDEO

GritAI YouTube

Video walkthroughs and practical demos of Claude Code workflows.

[Watch →](#)